

THE KNAPSACK FAMILY – SHARED SHAPE

```
// All three: for each item, either SKIP it or TAKE it, track a capacity/target.  
// dp[i][cap] = best answer using first i items with capacity/target cap  
// dp[i][cap] = combine(dp[i-1][cap] /*skip*/, dp[i-1][cap-w[i]] /*take*/)
```

0-1 KNAPSACK (each item used at most once)

Use when: maximize value under a weight limit, each item picked 0 or 1 times.

```
vector<vector<long long>> dp(n+1, vector<long long>(W+1, 0));  
for (int i=1;i<=n;i++) {  
    for (int cap=0;cap<=W;cap++) {  
        dp[i][cap] = dp[i-1][cap]; // skip item i  
        if (w[i-1] <= cap)  
            dp[i][cap] = max(dp[i][cap], dp[i-1][cap-w[i-1]] + val[i-1]); // take  
    }  
}  
// answer = dp[n][W]  
// 1D space-optimized: loop cap DESCENDING (prevents reusing item same pass)  
vector<long long> dp(W+1, 0);  
for (int i=0;i<n;i++) for (int cap=W;cap>=w[i];cap--) dp[cap]=max(dp[cap],dp[cap-  
w[i]]+val[i]);
```

COIN CHANGE (unlimited use of each coin)

Use when: fewest coins (or count ways) to make a target – coins reusable.

```
vector<int> dp(target+1, INT_MAX); // min coins to make each amount  
dp[0] = 0;  
for (int cap=1;cap<=target;cap++)  
    for (int c : coins) if (c<=cap && dp[cap-c]!=INT_MAX) dp[cap]=min(dp[cap],dp[cap-  
c]+1);  
// answer = dp[target] (or -1 if still INT_MAX)  
// NOTE: loop cap ASCENDING here – unlike knapsack, reuse is intended
```

SUBSET SUM (can a subset sum to exactly target?)

Use when: checking if some subset of elements sums to a target – true/false, each used once.

```
vector<bool> dp(target+1, false); dp[0] = true;  
for (int x : a)  
    for (int cap=target;cap>=x;cap--) dp[cap] = dp[cap] || dp[cap-x]; // descending:  
0-1 style  
// answer = dp[target]
```

QUICK COMPARISON

```
// 0-1 Knapsack : each item once      → loop capacity DESCENDING  
// Coin Change  : items reusable      → loop capacity ASCENDING  
// Subset Sum   : each item once, bool → loop capacity DESCENDING
```

===== HOW TO PICK THE RIGHT PATTERN =====

```
// "each item used at most once" + maximize value      → 0-1 Knapsack  
// "unlimited coins/items" + minimize count/ways        → Coin Change  
// "does a subset exist that sums to X" (true/false)   → Subset Sum  
// keyword "at most once" or "can't reuse"            → loop DESCENDING  
// keyword "unlimited" or "as many times as you want"  → loop ASCENDING
```

===== COMMON MISTAKE =====

```
// Using ascending loop for 0-1 Knapsack lets an item get counted twice  
// – always double check the loop direction matches the problem's reuse rule
```